

Working Updates

Site: [CVTE LMS](#)
Course: 170A WOBC/WOIC/WOAC Scripting
Book: Working Updates

Printed by: Brandon Larson,
Date: Friday, 24 April 2026, 7:59 AM

Table of contents

Chapter 1: PowerShell Scripting Foundations Review

- 1.1 – Objects, Output, and the Pipeline
- 1.2 – Help, Discovery, and Get-Member
- 1.3 – Variables, Strings, Arrays, Hashtables, Types, and Casting
- 1.4 – CIM Classes, Instances, and Property Retrieval
- 1.5 – Control Flow and Looping
- 1.6 – Functions, Parameters, and Execution Context
- 1.7 – Returning Structured Output with PSCustomObject
- 1.8 – Error Handling, Files, Paths, and Script Troubleshooting
- 1.9 – Chapter Review and Guided Practice
- 1.10 – Exercise

Chapter 2: .NET for PowerShell Practitioners

- 2.1 – PowerShell and .NET
- 2.2 – Properties, Methods, and Type Names
- 2.3 – Static vs Instance Members
- 2.4 – Useful .NET Types in Scripts
- 2.5 – Assemblies and Namespaces
- 2.6 – Practical Examples
- 2.7 – Chapter 2 Exercise

Chapter 3: Remote Execution, Security, and Operational Constraints

- 3.1 – Debugging Mindset
- 3.2 – WinRM and Remoting Basics
- 3.3 – Authentication, Delegation, and the Double-Hop Problem
- 3.4 – Common Remote Execution Issues
- 3.5 – Language Modes
- 3.6 – PowerShell Abuse and Defensive Context
- 3.7 – DSC, APIs, and Platform Integration
- 3.8 – Chapter 3 Exercise — Remote Connectivity Validator

Chapter 4: Performance and Parallel Execution

- 4.1 – Measuring Performance
- 4.2 – Improving Efficiency Before Parallelizing
- 4.3 – Jobs and Parallel Execution
- 4.4 – Scope, Imports, and Inputs in Jobs
- 4.5 – Parallel Patterns for This Course
- 4.6 – Chapter Review and Guided Practice

Chapter 5: Applied Remote Scripting Workshop

- 5.1 – Remote Query Pattern
- 5.2 – Build Function 1: Remote Scheduled Task Query

- 5.3 – Build Function 2: Remote Event Log Query
- 5.4 – Build Function 3: Deploy Remote Support Files
- 5.5 – Error Handling and Validation
- 5.6 – Module Assembly
- 5.7 – Orchestrator Pattern
- 5.8 – Parallel Execution from Ops
- 5.9 – Troubleshooting and Cleanup
- 5.10 – Chapter Review and Capstone Readiness

Chapter 6: Capstone

- 6.1 – Capstone Requirements
- 6.2 – Module Standard
- 6.3 – Required Functions
- 6.4 – Orchestrator Standard
- 6.5 – Parallel Execution Standard
- 6.6 – Error Handling Standard
- 6.7 – Validation and Testing Guide
- 6.8 – Submission Checklist

Chapter 1: PowerShell Scripting Foundations Review

💡 Chapter Purpose

This chapter refreshes the PowerShell knowledge and habits that make the rest of the course workable. The emphasis is on understanding what commands return, how to inspect and shape that data, how to use discovery tools effectively, and how to turn a working command into a small reusable function.

The course environment is built around a student workstation and multiple remote targets. Even when an example begins locally, it should be read with that larger execution model in mind.

System	Role in this course
Ops	Student workstation and execution host
System-1	Remote target
System-2	Remote target
System-3	Remote target
Depot	Remote target

Scripts will be developed and executed from Ops. As the course progresses, those scripts will gather information from or take action against System-1, System-2, System-3, and Depot. Chapter 1 therefore focuses on the core PowerShell patterns that make later remoting and orchestration reliable.

Learning Objectives

- Explain the difference between display output and the underlying object returned by a command. **(K1)**
- Use help and discovery commands to identify syntax, parameters, examples, properties, and methods. **(K1)**
- Use variables, strings, arrays, hashtables, type inspection, and type casting to support script development. **(K2)**
- Query Windows information with CIM, inspect classes and instances, and retrieve the properties that matter. **(K1)**
- Apply control flow, functions, structured output, and error handling to produce clean script behavior. **(K2)**
- Recognize how local execution context differs from intentionally remote behavior. **(K2)**

Chapter Map

Section	Focus
1.1	Objects, output, and the pipeline
1.2	Help, discovery, and Get-Member
1.3	Variables, strings, arrays, hashtables, types, and casting
1.4	CIM classes, instances, and property retrieval
1.5	Control flow and looping
1.6	Functions, parameters, and execution context
1.7	Returning structured output with PSCustomObject
1.8	Error handling, files, paths, and script troubleshooting
1.9	Chapter review and guided practice

✓ What matters most

By the end of this chapter, you should be able to read command output as data, inspect object members, pull useful properties from CIM, write a small function with parameters, and return a clean object that can still be used later in the pipeline.

1.1 – Objects, Output, and the Pipeline

PowerShell looks like a command-line shell, but it behaves differently from text-only shells in one important way: most commands return **objects**. Those objects carry named properties and methods. The screen shows a formatted view of the object, but the object itself usually contains more information than the display reveals.

That is why PowerShell can filter, sort, export, compare, and transform data without forcing you to parse strings. Once a returned object is understood, a command becomes much easier to script against.

Seeing the difference between display and data

Run a command that returns process objects and look at only a few properties first:

```
Get-Process | Select-Object ProcessName, Id, CPU -First 5
```

The output is shown as rows and columns, but those rows are not plain strings. Each row is a process object with many additional members that are not displayed by default.

Inspecting members with Get-Member

Whenever you need to know what an object contains, send it to `Get-Member`. This is one of the most useful habits in PowerShell.

```
Get-Service | Select-Object -First 1  
Get-Service | Select-Object -First 1 | Get-Member
```

The first line shows one returned service. The second line reveals the type, methods, and properties on that object. That tells you what can be selected, compared, exported, or stored in another object later.

Properties, methods, and types

A **property** holds data such as a service name or status. A **method** is an action you can call on the object. The **type** identifies what kind of object PowerShell is working with.

```
$Service = Get-Service -Name 'WinRM'  
$Service.Status  
$Service.DisplayName  
$Service.GetType().FullName
```

This pattern is foundational: run the command, save the result to a variable, and inspect the members you care about. Later chapters keep building on that same sequence.

 A common mistake

Formatting should happen near the end. Once you send objects to `Format-Table` or `Format-List`, you are no longer working with the original objects in a useful way. Filter and select first. Format last.

 Check your understanding

What does `Get-Member` tell you that the default table display does not?

1.2 – Help, Discovery, and Get-Member

PowerShell is much easier to work with once command discovery becomes part of the normal workflow. You do not need perfect recall before you begin. You need a repeatable way to find commands, inspect examples, and understand what the output contains.

Finding commands

`Get-Command` helps when you know part of a name, a noun, a verb, or the module that likely contains the command.

```
Get-Command -Verb Get
Get-Command -Noun Service
Get-Command -Name '*Cim*'
Get-Command -Module CimCmdlets
```

Reading help effectively

`Get-Help` is most useful when you move beyond the first screen. The examples and full help views are where command behavior usually becomes clear.

```
Get-Help Get-CimInstance
Get-Help Get-CimInstance -Examples
Get-Help Get-CimInstance -Full
```

The examples show intent. The full help view gives the parameter details, accepted input, notes, and related links. When a command matters to your script, both are worth checking.

Using Get-Member as a discovery tool

Help tells you how to use a command. `Get-Member` tells you what came back from the command. Both are needed.

```
Get-CimInstance -ClassName Win32_OperatingSystem | Get-Member
(Get-CimInstance -ClassName Win32_OperatingSystem).PSObject.Properties.Name
```

The first example shows the returned instance as an object. The second focuses only on the property names, which is a fast way to identify the fields that belong in a function or report.

 Discovery loop

A strong pattern is: find the command, read the examples, run the command once, inspect the returned object, then write the script.

1.3 – Variables, Strings, Arrays, Hashtables, Types, and Casting

Variables give a name to data so the value can be reused, changed, passed to another command, or tested in logic. PowerShell is flexible about variables, but scripts become more predictable when the underlying data type is understood.

Strings, integers, arrays, hashtables, and returned objects all behave differently. That difference affects comparison, looping, sorting, parameter binding, and output.

Working with common variable types

```
$ComputerName = 'System-1'
$Targets = @('System-1', 'System-2', 'System-3', 'Depot')
$Options = @{
    ClassName    = 'Win32_OperatingSystem'
    ErrorAction  = 'Stop'
}
```

An array holds multiple ordered values. A hashtable stores named key/value pairs. Both become important later when you loop through targets or splat parameters into commands.

Inspecting types

When a value behaves unexpectedly, check its type. That often explains the problem immediately.

```
$Port = '5985'
$Port.GetType().FullName
```

The value looks numeric, but it is still a string. If the script needs a numeric value, casting makes that intention explicit.

Casting values

```
$PortNumber = [int] $Port
$PortNumber.GetType().FullName
```

Type casting is not only for numbers. It is part of writing precise code. When a script is expected to receive text, return an integer, or compare dates properly, type awareness makes the behavior easier to reason through.

Keep this habit

Use `.GetType().FullName` when you want a quick answer about one value. Use `Get-Member` when you want to inspect the structure of a returned object.

1.4 – CIM Classes, Instances, and Property Retrieval

CIM stands for **Common Information Model**. In practical PowerShell work, CIM cmdlets are often used to retrieve structured information about Windows systems. The key idea is simple: a **class** defines what information is available, and an **instance** is the object returned with real values from a specific system.

Once that distinction is clear, `Get-CimClass` and `Get-CimInstance` fit naturally into the same PowerShell workflow you already use for other commands.

Inspecting a CIM class

```
Get-CimClass -ClassName Win32_OperatingSystem
Get-CimClass -ClassName Win32_OperatingSystem | Select-Object -ExpandProperty
CimClassProperties
```

This shows the schema-like side of CIM. It answers the question, “What could this class provide?” That is useful before you start selecting properties in a script.

Retrieving a CIM instance locally

```
$OS = Get-CimInstance -ClassName Win32_OperatingSystem
$OS.Caption
$OS.Version
$OS.BuildNumber
$OS.CSName
$OS.GetType().FullName
```

For a local query, the cleanest pattern is usually to omit `-ComputerName`. That keeps the query local and avoids turning a simple local retrieval into a remote-style connection attempt.

⚠ Local and remote are not the same thing

If a script passes the local computer name into `-ComputerName`, some environments still treat the call as a remote operation. For a local CIM query, leaving `-ComputerName` off is usually the cleanest pattern. Use it when you are intentionally targeting another host.

Retrieving a CIM instance remotely

```
Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName 'System-1'
```

The command changes very little. The class is the same. The output is still an object. The main difference is the target system and the environmental requirements needed to reach it.

Finding the properties you actually need

```
(Get-CimInstance -ClassName Win32_OperatingSystem).PSObject.Properties.Name  
Get-CimInstance -ClassName Win32_OperatingSystem | Select-Object Caption, Version,  
BuildNumber, LastBootUpTime
```

This is the point where raw retrieval becomes scripting. Once you know the class and its property names, you can choose the subset that belongs in a function, report, or validation result.

✓ Why CIM belongs here

The same basic skills apply: discovery, object inspection, property selection, variable assignment, and structured output. CIM is not a separate PowerShell language. It is another object source.

1.5 – Control Flow and Looping

Control flow determines how a script makes decisions and how often an action is repeated. Even simple scripts rely on this. A function may need one behavior for a local computer and another for a remote computer. A report may need to loop through several targets. A troubleshooting step may need to stop if a path does not exist.

Conditional logic with if and else

```
if ($ComputerName -eq $env:COMPUTERNAME) {  
    $OS = Get-CimInstance -ClassName Win32_OperatingSystem  
}  
else {  
    $OS = Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $ComputerName  
}
```

This kind of branching is small, but it makes script behavior much clearer. One path handles the local system. The other path handles an explicitly remote target.

Looping through several targets

```
$Targets = @('System-1', 'System-2', 'System-3', 'Depot')  
foreach ($Target in $Targets) {  
    Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $Target  
}
```

A `foreach` loop is often the cleanest way to apply the same action to multiple hosts. The current item is stored in a singular variable, which makes the body of the loop easier to read.

? Check your understanding

Which parts of the examples above change on each pass through the loop, and which parts stay the same?

1.6 – Functions, Parameters, and Execution Context

A function packages logic so it can be reused consistently. A good foundational function usually has a clear name, a small parameter block, and predictable output. It also behaves in a way that makes sense from the current execution context.

In this course, scripts execute from Ops. That means local work happens on Ops unless the function explicitly reaches to another system. Keeping that distinction in mind prevents a lot of early confusion.

A simple function pattern

```
function Get-SystemSnapshot {  
    param(  
        [string] $ComputerName = $env:COMPUTERNAME  
    )  
  
    if ($ComputerName -eq $env:COMPUTERNAME) {  
        $OS = Get-CimInstance -ClassName Win32_OperatingSystem  
    }  
    else {  
        $OS = Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $ComputerName  
    }  
    -ErrorAction Stop  
    $OS  
}
```

This pattern is useful because it gives the function a sensible default while still allowing a remote target to be passed in explicitly. It also avoids treating the local computer as a remote query when that is not necessary.

Execution context matters

If you run the function without parameters, it operates against the current host where the script runs. In this course, that is usually Ops. If you pass `System-1`, the function intentionally queries a different host. The code should make that distinction obvious.

✓ Build order that works well

Start by confirming the command locally. Then place it in a function. Then add the

parameter. Then add the local-versus-remote logic. Then add error handling. That sequence keeps troubleshooting manageable.

1.7 – Returning Structured Output with PSCustomObject

Returning the raw object is sometimes appropriate, especially while you are still exploring the data. In many scripts, however, it is more useful to return a smaller custom object that exposes only the values you want to keep. That makes the output easier to read and easier to reuse later.

Shaping the result

```
function Get-SystemSnapshot {  
    param(  
        [string] $ComputerName = $env:COMPUTERNAME  
    )  
  
    if ($ComputerName -eq $env:COMPUTERNAME) {  
        $OS = Get-CimInstance -ClassName Win32_OperatingSystem  
    }  
    else {  
        $OS = Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $ComputerName  
    }  
    -ErrorAction Stop  
  
    [pscustomobject]@{  
        ComputerName    = $OS.CSName  
        Caption         = $OS.Caption  
        Version         = $OS.Version  
        BuildNumber     = $OS.BuildNumber  
        LastBootUpTime  = $OS.LastBootUpTime  
    }  
}
```

This output is smaller, clearer, and easier to pass to other commands. It still behaves like an object, which means it can be sorted, filtered, exported, or collected into an array.

Using the returned object

```
$Result = Get-SystemSnapshot  
$Result | Format-List *  
$Result | Export-Csv -Path '.\snapshot.csv' -NoTypeInformation
```

Because the result is an object rather than display-only text, it stays useful after it leaves the function. That is one of the main reasons structured output matters so much in scripting.

Design choice

A well-designed function returns data in a form that is easy for the next command, the next function, or the next person to work with.

1.8 – Error Handling, Files, Paths, and Script Troubleshooting

Scripts become more reliable when they can recognize failure clearly. PowerShell does not treat every error the same way. Some errors are non-terminating, which means a command reports the issue but keeps going. Others are terminating, which means execution stops unless the error is handled.

Using try and catch effectively

A `catch` block responds to terminating errors. If the command writes only a non-terminating error, `catch` will not run unless the command is told to treat the error as terminating.

```
try {  
    Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName 'DoesNotExist' -  
    ErrorAction Stop  
}  
catch {  
    $_.Exception.Message  
}
```

The important detail here is `-ErrorAction Stop`. Without it, the command may write an error and continue instead of transferring control into `catch`.

Files and paths

Path handling matters because scripts often create logs, reports, or temporary files. Building paths explicitly is cleaner than relying on assumptions about the current working directory.

```
$ReportPath = Join-Path -Path $PSScriptRoot -ChildPath 'snapshot.csv'  
if (Test-Path $ReportPath) {  
    Remove-Item $ReportPath -Force  
}
```

`$PSScriptRoot` is especially useful in scripts because it refers to the folder where the script itself is stored. That makes file behavior more predictable.

A useful troubleshooting habit

When something fails, check the parameter values, the object type, the current execution context, and the exact error message before rewriting the function.

? Check your understanding

Why does adding -ErrorAction Stop change what try/catch can do?

1.9 – Chapter Review and Guided Practice

Chapter 1 establishes the foundation for the rest of the course. Commands return objects. Those objects expose properties and methods. Variables store values for reuse. Types explain behavior. Functions package logic. CIM provides another structured object source. Error handling and path management make that code safer to run repeatedly.

Key ideas to carry forward

- Use `Get-Member` to inspect returned objects before scripting against them.
- Use `Get-Command` and `Get-Help` as part of the normal build process.
- Use `.GetType().FullName` when the type of a value needs to be verified quickly.
- Use `Get-CimClass` to inspect a class and `Get-CimInstance` to retrieve actual data.
- For local CIM queries, omit `-ComputerName` unless the goal is specifically to perform a remote-style query.
- Use `[pscustomobject]` when you want to return a smaller, intentional set of properties.
- Use `-ErrorAction Stop` when a failing command must be caught by `catch`.

Guided practice sequence

1. Run a command such as `Get-Service` and inspect the returned object with `Get-Member`.
2. Store that result in a variable and retrieve two properties directly.
3. Use `Get-CimClass` and `Get-CimInstance` to explore `Win32_OperatingSystem`.
4. Write a small function that accepts `$ComputerName` with a default of `$env:COMPUTERNAME`.
5. Return a custom object that includes computer name, caption, version, and build number.
6. Add `try/catch` and `-ErrorAction Stop` to handle an invalid remote target cleanly.

Readiness check

If you can complete the guided practice sequence without guessing what the command returned or why it behaved a certain way, you are in a strong position to move into the next chapter.

1.10 – Exercise

Chapter 1 Exercise — Build a System Info Reporter

Put together the skills from this chapter into one short script. Work from Ops.

1. Write a function called `Get-SystemSnapshot` that accepts a `-ComputerName` parameter with a default value of `$env:COMPUTERNAME`.
2. Inside the function, use `Get-CimInstance Win32_OperatingSystem` to retrieve the OS caption and last boot time.
3. Return a `[pscustomobject]` with properties: **ComputerName**, **OSCaption**, **LastBoot**, **Status**, and **Error**.
4. Wrap the CIM query in a `try/catch` block. On failure, return a failure object with the exception message.
5. Call your function twice — once with no argument (`local`), once with an intentionally bad computer name — and pipe both results to `Format-Table`.

Success criteria: Both the success and failure rows display cleanly. The error message is captured in the object, not thrown to the console.

Chapter 2: .NET for PowerShell Practitioners

PowerShell does not just resemble .NET — it *is* a .NET application. Every object that flows through the pipeline, every type cast, every `Get-Member` result comes from .NET. This chapter makes that relationship explicit so the patterns you already use become easier to reason about and extend.

Chapter Purpose

This chapter focuses on the .NET concepts that show up inside PowerShell scripting. The goal is not to turn you into a .NET developer. The goal is to help you recognize the object and type patterns that make PowerShell more powerful.

Learning Objectives

- Explain how .NET types appear in PowerShell output and commands. (K1)
- Inspect object properties and methods by using discovery tools. (K1)
- Create structured output by using `[pscustomobject]`. (K1)
- Distinguish between static member syntax and instance method usage. (K1)
- Apply practical .NET patterns commonly used in PowerShell scripts. (K1)

Chapter Map

Section	Focus
2.1	PowerShell and .NET
2.2	Properties, methods, and type names
2.3	Static vs instance members
2.4	Useful .NET types in scripts
2.5	Assemblies and namespaces
2.6	Practical examples
2.7	Chapter exercise

What matters most

By the end of this chapter, you should be able to read command output as data, inspect

object members, pull useful properties from CIM, write a small function with parameters, and return a clean object that can still be used later in the pipeline.

2.1 – PowerShell and .NET

PowerShell is built on top of the .NET runtime. This is not trivia — it is the reason PowerShell commands can return rich objects instead of plain text. Every object you work with in PowerShell is a .NET object with a type name, properties, and methods.

You can see which PowerShell version and edition you are running:

```
$PSVersionTable
```

The **PSVersion** line shows the PowerShell version. The **PSEdition** line shows whether you are running Windows PowerShell (**Desktop** , built on .NET Framework) or PowerShell 7+ (**Core** , built on modern .NET). These two editions behave differently enough that the distinction matters when a script works on one host and fails on another.

In Windows PowerShell 5.1, a **CLRVersion** property also appears in **\$PSVersionTable** and reports the .NET Framework version. That property is not present in PowerShell 7. If you need the exact runtime version on PowerShell 7, use:

```
[System.Runtime.InteropServices.RuntimeInformation]::FrameworkDescription
```

Every object you see in PowerShell output — processes, services, dates, file info — is backed by a .NET class. The **PSEdition** you are running determines which .NET classes are actually available, which is one reason a script can succeed on one host and fail on another.

Why this matters

When you pipe **Get-Date** to **Get-Member** , you are looking at the .NET type **System.DateTime** and its members. Understanding this connection helps you find capabilities that are not exposed as standalone cmdlets. Understanding which *edition* of .NET you are on helps you avoid writing scripts that work on your workstation but fail on a PS 5.1 host — or vice versa.

2.2 – Properties, Methods, and Type Names

Properties describe data — they tell you what an object has. Methods perform actions — they tell you what an object can do. Type names tell you what kind of object you are working with.

Discovering a type name

```
$Date = Get-Date  
$Date.GetType().FullName
```

Listing properties and methods

```
$Date | Get-Member -MemberType Property  
$Date | Get-Member -MemberType Method
```

Accessing properties vs calling methods

Properties use dot notation with no parentheses. Methods use dot notation with parentheses.

```
$Date.DayOfWeek          # property -- no parentheses  
$Date.AddDays(7)         # method -- parentheses required
```

⚠ Common mistake — calling a method without parentheses

If you write `$Date.AddDays` without parentheses, PowerShell returns a description of the method instead of calling it. Always include parentheses when calling a method.

? Check your understanding

When you inspect an object with `Get-Member` and see a member labeled Property vs one labeled Method, how do you know which syntax to use?

2.3 – Static vs Instance Members

An instance member is called on a specific object — you need an object first. A static member is called on the type itself — no object needed.

Instance members — need an object

```
$Date = Get-Date           # create an instance first
$Date.AddDays(1)           # call method on that instance
$Date.DayOfWeek            # access property on that instance
```

Static members — called on the type with ::

```
[datetime]::Now            # no object needed
[environment]::MachineName # no object needed
[environment]::UserName    # no object needed
```

The double-colon `::` is the static member operator. When you see it in a script, you know the member belongs to the type, not to an instance.

? Check your understanding

Which of the examples above require an existing object in a variable, and which do not?

2.4 – Useful .NET Types in Scripts

A few .NET types show up frequently in PowerShell scripting. You do not need to memorize the full .NET class library — just know these common ones.

Type	What it gives you	Example
<code>[datetime]</code>	Timestamps, date math	<code>[datetime]::Now</code>
<code>[environment]</code>	Machine name, user, OS info	<code>[environment]::MachineName</code>
<code>[System.IO.Path]</code>	Path manipulation	<code>[System.IO.Path]::Combine('C:\Tools', 'Sysmon')</code>
<code>[math]</code>	Rounding, max/min	<code>[math]::Round(3.14, 2)</code>
<code>[System.Net.Dns]</code>	DNS lookups	<code>[System.Net.Dns]::GetHostName()</code>

```
# Build a timestamp for logging
[datetime]::Now.ToString('yyyy-MM-dd HH:mm:ss')

# Safe path joining
[System.IO.Path]::Combine('C:\Tools', 'Sysmon', 'Sysmon64.exe')

# Get the current machine name
[environment]::MachineName
```

Namespace shorthand

PowerShell accepts namespace shorthand for common types: `[IO.Path]` resolves to `[System.IO.Path]`, `[Net.Dns]` resolves to `[System.Net.Dns]`. Both forms work. This course uses the full name in reference material and the shorthand when space is tight in a script.

2.5 – Assemblies and Namespaces

Assemblies package compiled .NET code. Namespaces organize types into groups. In most day-to-day scripting, you will use built-in types without needing to think about this.

```
try {  
    Add-Type -AssemblyName System.Windows.Forms -ErrorAction Stop  
    'Assembly loaded successfully'  
}  
catch {  
    $_.Exception.Message  
}
```

✓ Course scope

Assemblies and namespaces are background knowledge for this course. If you find yourself deep in .NET assembly loading, you have probably overcomplicated the solution.

2.6 – Practical Examples

The best use of .NET in PowerShell scripts is when it makes the code easier to read or when PowerShell already exposes the type naturally.

```
[pscustomobject]@{  
    Timestamp    = [datetime]::Now  
    Workstation  = [environment]::MachineName  
    UserName     = [environment]::UserName  
    SysmonPath   = [System.IO.Path]::Combine('C:\Tools', 'Sysmon', 'Sysmon64.exe')  
}
```

When NOT to use .NET directly

If a cmdlet already does the job, prefer the cmdlet. It is more readable and more discoverable.

```
# Prefer this:  
Test-Path 'C:\Tools\Sysmon\Sysmon64.exe'  
  
# Over this (same result, harder to read):  
[System.IO.File]::Exists('C:\Tools\Sysmon\Sysmon64.exe')
```

2.7 – Chapter 2 Exercise

Scenario

You need to create a script that audits the current execution environment. It must identify the underlying PowerShell version, calculate a "Review Date" 30 days into the future, and identify the current user and machine.

Exercise Requirements

Using the .NET discovery and application skills from this chapter, complete the following tasks:

- **Discovery:** Identify the full .NET type name of the `$PSVersionTable` object.
- **Static Members:** Use `[datetime]` to get the current time and `[environment]` to get the current username.
- **Instance Methods:** Take the current date object and use a method to calculate a date exactly 30 days in the future.
- **Math:** Use the `[math]` type to round the number `124.76` to the nearest whole integer.
- **Output:** Return all these findings in a single `[pscustomobject]`.

Exercise Skeleton (`Test-Environment.ps1`)

```
# 1. Discover the type of $PSVersionTable
$TableType = #

# 2. Get current system info using static members (::)
$Now = #
$User = #

# 3. Calculate a review date using an instance method (.)
$ReviewDate = #

# 4. Use [math] to round 124.76
$RoundedValue = #

# 5. Return the results as a structured object
[pscustomobject]@{
    NET_Type      = $TableType
    CurrentUser   = $User
    ReviewDue     = $ReviewDate
    RoundedNum    = $RoundedValue
}
```

Chapter 3: Remote Execution, Security, and Operational Constraints

Chapter 3: Remote Execution, Security, and Operational Constraints

Chapter 3: Remote Execution, Security, and Operational Constraints

Chapter Purpose

This chapter covers the operational knowledge you need before orchestrating work across remote systems. The focus is on remoting realities, debugging mindset, security context, and common constraints. Several topics — DSC, APIs, Azure — are conceptual rather than hands-on.

Learning Objectives

- Apply an organized approach to debugging PowerShell scripts. **(K2)**
- Explain common remote execution issues, including authentication and delegation concerns. **(K3, K8)**
- Describe the double-hop problem at a practical level. **(K3)**
- Recognize how PowerShell language modes change available behavior. **(K4)**
- Compare common attacker techniques using PowerShell and describe how defenders respond. **(K6)**
- Describe the role of DSC, API interaction patterns, and platform integration at a knowledge level. **(K7, K10, K11)**

Chapter Map

Section	Focus
3.1	Debugging mindset
3.2	WinRM and remoting basics
3.3	Authentication, delegation, and double-hop
3.4	Common remote execution issues
3.5	Language modes
3.6	PowerShell abuse and defensive context

Section	Focus
3.7	DSC, APIs, and platform integration
3.8	Chapter review and scenario questions

3.1 – Debugging Mindset

Chapter 3 › 3.1 – Debugging Mindset

3.1 – Debugging Mindset

Debugging is not about random changes. It is about **controlled observation**. When a script fails, determine what failed, where it failed, and what input triggered it.

The four-step approach

1. **Read the exact error message.** Not the gist — the actual text.
2. **Confirm the execution context.** Where is the command running? As which user?
3. **Test assumptions one at a time.** If you change five things at once, you will not know which fix solved it.
4. **Reduce to the failing command.** Run it alone in the console.

```
# Step 2 – check execution context
whoami
$env:COMPUTERNAME
Get-Location

# Step 3 – test one assumption
Test-Path 'C:\Tools\Sysmon\Sysmon64.exe'
```

✓ Practical standard

Do not change five things at once. Verify one assumption at a time.

3.2 – WinRM and Remoting Basics

Chapter 3 › 3.2 – WinRM and Remoting Basics

3.2 – WinRM and Remoting Basics

PowerShell remoting uses **Windows Remote Management (WinRM)** as its transport. In this course, you execute from Ops and remotely interact with System-1, System-2, System-3, and Depot.

Testing connectivity

```
Test-WSMan System-1
```

Three remoting patterns

Pattern	Use case	Command
Interactive session	Manual exploration	Enter-PSSession
One-shot command	Run a single script block	Invoke-Command
Persistent session	Multi-step operations (copy + execute)	New-PSSession + Copy-Item - ToSession

```
# One-shot
Invoke-Command -ComputerName 'System-1' -ScriptBlock { hostname }

# Persistent session
$Session = New-PSSession -ComputerName 'System-1'
Invoke-Command -Session $Session -ScriptBlock { hostname }
Remove-PSSession $Session
```

Using local variables inside a remote script block

A remote script block runs on the target system, not in the local Ops session. That means variables created on Ops are not automatically available inside the remote command. When a remote command needs a local value, use the `$using:` scope modifier to copy that value into the remote script block at execution time.

```
$Target = 'System-1'
$ServiceName = 'WinRM'

Invoke-Command -ComputerName $Target -ScriptBlock {
    Get-Service -Name $using:ServiceName
}
```

Without `$using:ServiceName`, the remote session looks for a variable named `$ServiceName` on System-1. Since that variable was created on Ops, the remote session does not know about it. Use `$using:` when the value already exists locally and the remote command only needs to read it.

Keep the direction straight

`$using:` sends a local value into the remote script block. It does not create a permanent variable on the remote computer, and it does not pull a remote variable back to Ops.

Course standard

PSSession remoting is the approved remote execution pattern for the capstone deployment function.

3.3 – Authentication, Delegation, and the Double-Hop Problem

Chapter 3 › 3.3 – Authentication, Delegation, and the Double-Hop Problem

3.3 – Authentication, Delegation, and the Double-Hop Problem

The **double-hop problem** occurs when credentials used for the first remote connection are not automatically delegated to a second resource from inside that remote session.

How it works

Hop 1: Ops → System-1 (succeeds — your credentials are presented)

Hop 2: System-1 → Depot (fails — System-1 does not have your credentials to forward)

The confusing part: the *same command works fine when run directly on System-1* — because there is no second hop when you are already on the machine.

Why you need to know this

In the capstone, your deployment function uses `Copy-Item -ToSession`. This avoids the double-hop because the file transfer is initiated from Ops, not from inside the remote session.

Common ways to handle it

In production environments, double-hop issues are usually solved with an intentional delegation design, not by guessing at credentials. Common options include **CredSSP**, Kerberos constrained delegation, resource-based constrained delegation, or a hub-and-spoke copy pattern where Ops pushes files and commands to each target directly. CredSSP is a common workaround because it allows the remote system to use delegated credentials, but that also increases risk because credentials are made available to the remote host. For this course and the capstone, prefer the safer copy-from-Ops pattern unless an instructor explicitly tells you to test a delegation-based approach.

💡 Why this matters

A script may work locally and even remotely, but still fail when a second remote resource is involved. Understanding the authentication path helps you diagnose the real issue.

? Check your understanding

In the capstone deployment pattern, which machine initiates the file copy — Ops or the remote target? Why does this matter?

3.4 – Common Remote Execution Issues

Chapter 3 › 3.4 – Common Remote Execution Issues

3.4 – Common Remote Execution Issues

Symptom	Likely cause	First check
WinRM cannot connect	WinRM not enabled or firewall blocking	Test-WSMan [target]
Access denied	Wrong credentials or insufficient permissions	See credential format callout below
Command not found remotely	Module not installed on target	Run <code>Get-Command</code> inside remote session
Path not found	Path exists on Ops but not on target	Invoke-Command { Test-Path ... }
Silently wrong results	Execution context differs	Run <code>whoami</code> inside remote session
Works locally, fails in job	Variable scope — parent vars not inherited	Use <code>-ArgumentList</code>

```
# Quick connectivity and identity check
$Cred = Get-Credential
Invoke-Command -ComputerName 'System-2' -Credential $Cred -ScriptBlock {
    <span style="color: #e5c07b;">whoami</span>
    <span style="color: #e5c07b;">hostname</span>
}
```

⚠ Common mistake — wrong credential format

In this domain-based range, enter credentials in one of two formats:

- **NetBIOS:** [DOMAIN]\[user] — for example, if the domain is named CVTELAB and the user is student01, enter CVTELAB\student01.
- **UPN:** [user]@[domain.fqdn] — for example, student01@cvtelab.local.

If you enter just `student01` with no domain qualifier, Windows resolves it against the local account database on your workstation, not the domain. You will either authenticate as a local account (wrong identity) or fail outright.

Both NetBIOS and UPN formats work with `Get-Credential`, `New-PSSession -Credential`, and `Invoke-Command -Credential`. If one format fails unexpectedly, try the other before assuming a permissions problem.

3.5 – Language Modes

Chapter 3 › 3.5 – Language Modes

3.5 – Language Modes

PowerShell **language modes** control what a session is allowed to do. They are a security mechanism that restricts script capabilities.

Mode	What it allows	When you see it
FullLanguage	Everything — .NET, Add-Type, all cmdlets	Default on most workstations
ConstrainedLanguage	Basic cmdlets and approved types only	AppLocker or WDAC-enforced environments
RestrictedLanguage	Very limited — no script blocks	Rare; specific configurations
NoLanguage	Commands only, no scripting	Extremely locked-down endpoints

Checking the current mode

```
$ExecutionContext.SessionState.LanguageMode
```

Why this matters

If a script behaves differently on one host than another, language mode is one of the first things to check.

Check your understanding

If System-3 is running in ConstrainedLanguage mode, which of the .NET techniques from Chapter 2 would stop working?

3.6 – PowerShell Abuse and Defensive Context

Chapter 3 › 3.6 – PowerShell Abuse and Defensive Context

3.6 – PowerShell Abuse and Defensive Context

PowerShell is widely used for administration and automation. It is also one of the most commonly abused tools by adversaries.

Why attackers use PowerShell

- **Already installed** — no need to drop a binary.
- **Runs in memory** — encoded commands execute without writing files to disk.
- **Deep system access** — WMI, .NET, COM objects, registry, Active Directory.
- **Remote execution** — WinRM and PSRemoting enable lateral movement.

Common attack patterns

Technique	What it looks like
Encoded commands	<code>powershell -EncodedCommand</code>
Download cradles	<code>IEX (New-Object Net.WebClient).DownloadString(...)</code>
Living off the land	Using built-in cmdlets for recon and lateral movement
AMSI bypass	Attempting to disable Antimalware Scan Interface

Defensive controls

Control	What it does
Script Block Logging	Records full content of every script block executed
Transcription	Captures all console input and output
Constrained Language Mode	Restricts .NET access
AMSI	Scans script content at runtime
Code signing	Restricts which scripts can run

 Why this matters

As a 170A, you should understand both the legitimate value of PowerShell and why unusual usage may require investigation. The same remoting and scripting skills you use in the capstone are the same capabilities adversaries exploit.

3.7 – DSC, APIs, and Platform Integration

Chapter 3 › 3.7 – DSC, APIs, and Platform Integration

3.7 – DSC, APIs, and Platform Integration

This section covers three topics at a conceptual level. You will not build scripts around these for the capstone, but you should recognize them for the written assessment.

Desired State Configuration (DSC)

DSC is a PowerShell-driven model for defining how systems *should* be configured, then automatically enforcing that state. You declare the desired state and let the DSC engine handle enforcement.

```
# Conceptual DSC example – not a hands-on exercise
Configuration BaselineConfig {
    Node 'System-1' {
        Service WinRM {
            Name      = 'WinRM'
            State      = 'Running'
            StartupType = 'Automatic'
        }
    }
}
```

API Interactions

PowerShell can interact with web APIs using `Invoke-RestMethod`.

```
$Response = Invoke-RestMethod -Uri 'https://jsonplaceholder.typicode.com/users/1' -Method
Get
$Response.name
$Response.email
```

Azure and Cloud Integration

PowerShell integrates with Microsoft Azure through the `Az` module. Commands like `Connect-AzAccount` and `Get-AzVM` allow you to manage cloud resources.

 Course scope

DSC, API interactions, and Azure integration are conceptual topics. You will encounter them on the written assessment at a recognition level.

3.8 – Chapter 3 Exercise — Remote Connectivity Validator

 **Scenario:** Before orchestrating work across multiple systems in your capstone, you must verify the operational constraints of the environment. This exercise confirms connectivity and identifies security restrictions like Language Modes.

Part 1: The Connectivity Validator

Confirm reachability and identity across all course targets.

- **Looping:** Create an array for 'System-1', 'System-2', 'System-3', 'Depot' .
- **Connectivity:** Use `Test-WSMan` inside a `try/catch` block.
- **Remote Context:** Use `Invoke-Command` to gather `whoami` and the session's **LanguageMode**.

Part 2: Deployment Logic Auditor (Capstone Prep)

Practice the persistent session and scoping patterns required for the Sysmon deployment.

- **Persistence:** Open a persistent session to `System-1` using `New-PSSession` .
- **Variable Scoping:** Define a local variable (e.g. `$LocalMsg`) and use the **\$using:** scope modifier to display it inside the remote session.
- **The "Push" Pattern:** Practice using `Copy-Item -ToSession` to move a small test file from `Ops` to `C:\Temp` .

Exercise Skeleton

```
# Define Targets
$Targets = @('System-1', 'System-2', 'System-3', 'Depot')

# Part 1: Connectivity Validator
$Results = foreach ($Target in $Targets) {
    try {
        # [Check WinRM connectivity]
        # [Gather remote context via Invoke-Command]
        # [Return [pscustomobject]]
    }
    catch {
        # [Handle failure]
    }
}
$Results | Format-Table

# Part 2: Deployment Scoping (System-1)
# [Open session, test $using: scope, and clean up]
```

Chapter 4: Performance and Parallel Execution

💡 Chapter Purpose

This chapter develops the performance habits and job-management patterns that make large-scale execution practical. The focus is not speed for its own sake. The focus is understanding when work can be reduced, when concurrency is appropriate, and how to coordinate parallel tasks from Ops without losing control of inputs, imports, or results.

By the time Chapter 4 is complete, performance should feel like a design consideration rather than a cleanup task. A script that works once is useful. A script that works across multiple targets, returns structured results, and finishes in a predictable amount of time is far more valuable.

This chapter also serves as the bridge to the orchestrator. The later capstone does not ask for every possible concurrency technique in PowerShell. It asks for a clean, reliable pattern that can coordinate work across the course targets from **Ops**.

System	Role in this chapter
Ops	Execution host where jobs are created, monitored, and collected
System-1	Remote target
System-2	Remote target
System-3	Remote target
Depot	Remote target

Learning Objectives

- Explain why runtime, scale, and coordination matter when one script must operate across several targets.
- Measure script execution time and interpret the result in a meaningful way.
- Improve efficiency before introducing concurrency.
- Use PowerShell jobs to run independent work in parallel.
- Explain how jobs affect scope, imports, arguments, and result collection.
- Apply one clear parallel pattern that can be reused later in the course.

Chapter Map

Section	Focus
4.1	Measuring performance
4.2	Improving efficiency before parallelizing
4.3	Jobs and parallel execution
4.4	Scope, imports, and inputs in jobs
4.5	Parallel patterns for this course
4.6	Chapter review and guided practice

4.1 – Measuring Performance

Performance work begins with one discipline: measure the script that actually exists before deciding how to improve it. That sounds simple, but it prevents a common mistake. A script can feel slow because it touches several targets, because it repeats the same work unnecessarily, or because it introduces overhead that is easy to miss when only one command is tested in isolation.

For this course, the most useful measurement is usually **wall-clock time**: how long the whole operation takes from start to finish. That is the number that matters when Ops must coordinate work across System-1, System-2, System-3, and Depot.

Using Measure-Command well

Measure-Command wraps a block of code, runs it, and returns timing data. It does not change the script. It simply gives the script a stopwatch.

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'

$Timing = Measure-Command {
    foreach ($ComputerName in $Targets) {
        Invoke-Command -ComputerName $ComputerName -ScriptBlock {
            Get-Service -Name WinRM |
            Select-Object MachineName, Status, StartType
        }
    }
}

$Timing.TotalSeconds
```

This example measures the total time required to query the same service on four targets from Ops. The returned value is not the service data. It is a **TimeSpan** object that describes how long the operation took.

```
$Timing.GetType().FullName
$Timing | Select-Object Days, Hours, Minutes, Seconds, Milliseconds, TotalSeconds
```

That extra inspection matters. Looking only at a formatted value can hide useful properties. The object tells the full story.

What a measurement does and does not mean

One timing result is informative, but it is not final. Remote work can vary based on connection state, current load, caching, and network conditions. The right habit is to compare runs under the same conditions and look for patterns rather than assuming one number is absolute.

```
1..3 | ForEach-Object {  
    Measure-Command {  
        foreach ($ComputerName in $Targets) {  
            Invoke-Command -ComputerName $ComputerName -ScriptBlock {  
                Get-Service -Name WinRM | Select-Object MachineName, Status  
            }  
        }  
    } | Select-Object TotalSeconds  
}
```

Repeated measurements help answer a more useful question: is the script consistently slow, or was one run unusual?

Reading the result

Use a measurement to guide the next design choice. A timing result by itself does not prove that jobs are required. It only shows how long the current approach takes.

Sequential work is often the baseline

Before any parallel pattern is introduced, it is helpful to establish a clear baseline. When a script runs sequentially, the flow is easy to understand: one target finishes, then the next begins. That predictability makes sequential execution the right place to start even when the long-term goal is concurrency.

In the next section, the goal is not to jump straight to parallel execution. The goal is to remove waste first.

4.2 – Improving Efficiency Before Parallelizing

Parallel execution is powerful, but it does not fix inefficient logic. A script that repeats unnecessary work will usually remain wasteful even when that work is spread across several jobs. The cleanest performance gains often come from improving the design before introducing concurrency.

Reduce work before adding workers

Several common issues increase runtime without adding value:

- reading the same data repeatedly inside a loop
- filtering late instead of early
- building remote calls around information that could have been prepared once on Ops
- collecting screen output instead of returning usable objects

The general rule is simple: perform shared setup once, then reuse it.

```
# Less efficient: repeated setup around the real task
foreach ($ComputerName in $Targets) {
    $SelectedService = Get-Service | Where-Object Name -eq 'WinRM'
    Invoke-Command -ComputerName $ComputerName -ScriptBlock {
        Get-Service -Name WinRM | Select-Object MachineName, Status
    }
}

# More efficient: define the intent once, then reuse it
$ServiceName = 'WinRM'
foreach ($ComputerName in $Targets) {
    Invoke-Command -ComputerName $ComputerName -ScriptBlock {
        param($ServiceName)
        Get-Service -Name $ServiceName | Select-Object MachineName, Status
    } -ArgumentList $ServiceName
}
```

The improvement is not dramatic because the example is small, but the principle scales. Repeating work that can be done once is one of the easiest ways to slow down a larger script.

Filter and shape early

Another useful habit is to reduce the amount of data carried forward. If only a few properties matter, select them. If only a subset of targets should be processed, determine that before the heavy work begins.

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$TargetsToQuery = $Targets | Where-Object { $_ -ne 'Depot' }

foreach ($ComputerName in $TargetsToQuery) {
    Invoke-Command -ComputerName $ComputerName -ScriptBlock {
        Get-Service -Name WinRM |
        Select-Object MachineName, Status, StartType
    }
}
```

That approach makes the intent clearer and reduces unnecessary work. The same idea applies to objects: return the properties that matter, not every property available by default.

Efficiency improves readability too

Well-structured scripts are usually easier to read. Inputs are defined once. Filters are visible. Remote actions are deliberate. Performance and readability often improve together because both benefit from clear design.

Checkpoint

Before using jobs, confirm that the script is already doing only the work that needs to be done. Parallelizing waste is still waste.

4.3 – Jobs and Parallel Execution

Once the script has a clean sequential design, the next question is whether the units of work are independent. If each target can be processed without depending on the result from another target, the task is a good candidate for parallel execution.

In PowerShell, a job is a background task that runs separately from the foreground prompt. Jobs make it possible to start multiple operations, continue working, then collect the results later.

Basic job lifecycle

```
$Job = Start-Job -ScriptBlock {  
    Get-Date  
}  
  
Get-Job  
Wait-Job -Job $Job  
Receive-Job -Job $Job  
Remove-Job -Job $Job
```

That pattern is worth memorizing. Jobs are started, monitored, waited on, received, and removed. Leaving jobs behind creates clutter and makes later troubleshooting harder.

Applying jobs to multiple targets

The more useful pattern for this course is one job per target. Ops launches the jobs. Each job performs the same kind of work against a different remote system.

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'

$Jobs = foreach ($ComputerName in $Targets) {
    Start-Job -Name $ComputerName -ArgumentList $ComputerName -ScriptBlock {
        param($ComputerName)

        Invoke-Command -ComputerName $ComputerName -ScriptBlock {
            Get-Service -Name WinRM |
            Select-Object MachineName, Status, StartType
        }
    }
}

$null = $Jobs | Wait-Job
$Results = $Jobs | Receive-Job
$Jobs | Remove-Job

$Results
```

This pattern reduces total wall-clock time when the work is truly independent and the overhead of running jobs is justified. The gain is usually seen at the whole-run level, not in the speed of an individual query.

When jobs help and when they do not

Jobs are not automatically faster. Very small or very short tasks may complete so quickly that the extra overhead adds little value. Jobs are most helpful when there are several similar units of work that can be started and collected independently.

Good candidate for jobs

Processing the same action across System-1, System-2, System-3, and Depot is a good fit because each target can be handled independently.

4.4 – Scope, Imports, and Inputs in Jobs

The most important detail about jobs is that they run in a separate session context. A function available in the current console may not exist inside the job. A variable visible in the current script may not be present inside the job. A module imported earlier may need to be imported again inside the job.

Most job failures in this course come from that separation. The command works in the console, then fails in the background because the job cannot see the same state.

Passing values into a job

Use parameters and arguments to pass required values into the job script block deliberately.

```
$ComputerName = 'System-1'
$ServiceName  = 'WinRM'

Start-Job -ArgumentList $ComputerName, $ServiceName -ScriptBlock {
    param($ComputerName, $ServiceName)

    Invoke-Command -ComputerName $ComputerName -ScriptBlock {
        param($ServiceName)
        Get-Service -Name $ServiceName | Select-Object MachineName, Status
    } -ArgumentList $ServiceName
}
```

That design makes the inputs obvious and avoids hidden dependencies.

Importing a module inside the job

If a job needs a function stored in a module, import that module inside the job. Do not assume that importing it in the parent session is enough.

```
$ModulePath = 'C:\Tools\student_JobExamples.psm1'
$ComputerName = 'System-1'

Start-Job -ArgumentList $ModulePath, $ComputerName -ScriptBlock {
    param($ModulePath, $ComputerName)

    Import-Module $ModulePath -Force
    Get-RemoteWinRMStatus -ComputerName $ComputerName
}
```


The specific function name will vary later in the course, but the pattern remains the same: pass the module path, import it inside the job, then call the function.

Returning objects instead of screen-only text

Jobs are much easier to work with when they return structured objects. A formatted string may look readable, but it is harder to sort, filter, export, and validate later.

```
Start-Job -ArgumentList $ComputerName -ScriptBlock {  
    param($ComputerName)  
  
    [pscustomobject]@{  
        ComputerName = $ComputerName  
        Reachable    = $true  
        Timestamp    = Get-Date  
    }  
}
```

That result can be collected, combined, and exported with very little cleanup.

Troubleshooting order

When a job fails unexpectedly, first verify the inputs, then verify the module or function availability inside the job, then inspect the returned errors before changing the larger design.

4.5 – Parallel Patterns for This Course

At this point the chapter narrows to one course pattern: **Ops defines the targets, starts one job per target, waits, receives, and cleans up.** That is the parallel pattern that matters most for the later orchestrator.

Canonical course pattern

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$ModulePath = 'C:\Tools\student_JobExamples.psm1'

$Jobs = foreach ($ComputerName in $Targets) {
    Start-Job -Name $ComputerName -ArgumentList $ModulePath, $ComputerName -ScriptBlock {
        param($ModulePath, $ComputerName)

        Import-Module $ModulePath -Force

        Get-RemoteWinRMStatus -ComputerName $ComputerName
    }
}

$null = $Jobs | Wait-Job
$Results = $Jobs | Receive-Job
$Jobs | Remove-Job

$Results
```

Even though the specific function later changes, the orchestration logic stays familiar. The calling script coordinates the run. The module does the task-specific work. That division keeps the design easier to read and easier to test.

Why this pattern works well for the course

- the workflow remains centralized on Ops
- each target becomes one independent unit of work
- results are gathered into one collection after completion
- module-based logic can be reused without rewriting the orchestrator

This is also where discipline matters. The pattern is simple on purpose. It is better to use one clean, dependable pattern well than to switch between several valid but inconsistent approaches.

Design standard

The orchestrator should coordinate the run. The module should perform the target-specific work.

Sequential and parallel designs can coexist

Parallel execution is useful when the work benefits from it. Sequential execution is still valuable for initial testing, troubleshooting, or small-scale validation. A dependable workflow often starts sequentially, proves the logic, and then adopts the parallel wrapper once the task logic is stable.

4.6 – Chapter Review and Guided Practice

Chapter 4 turns performance into a practical design habit. Measure the current approach. Remove avoidable waste. Introduce jobs only when the work is independent. Pass inputs deliberately. Import what the job needs. Return structured results. Clean up when the work is complete.

Key ideas to carry forward

- Use `Measure-Command` to establish a real baseline before changing the design.
- Reduce unnecessary work before parallelizing.
- Use jobs when multiple targets can be processed independently.
- Remember that jobs run in a separate context and may need their own imports and inputs.
- Return objects so the orchestrator can evaluate and reuse results later.
- Remove completed jobs to keep the session clean.

Guided practice

1. Measure a sequential query from Ops against all four targets and record the total runtime.
2. Build a one-job-per-target version of the same task and compare the total runtime.
3. Modify the job so that the service name is passed in as an argument instead of being hardcoded.
4. Create a small custom object inside the job and return it through `Receive-Job`.
5. Deliberately leave jobs uncleared once, inspect them with `Get-Job`, then clean them up properly.

Review questions

1. Why is **Measure-Command** useful before introducing jobs?
2. What kinds of tasks are good candidates for one-job-per-target execution?
3. Why can a function work in the console but fail inside a job?
4. Why is returning an object usually better than returning a formatted string?
5. What are the standard steps in the job lifecycle after a set of jobs has been created?

Looking ahead

The next chapters build on this pattern by shifting attention from generic job control to reusable task logic, orchestration design, and the capstone workflow.

Chapter 5: Applied Remote Scripting Workshop

Chapter 5: Applied Remote Scripting Workshop

Chapter 5: Applied Remote Scripting Workshop

Chapter Purpose

This is the build chapter. You will construct three working functions and an orchestrator. The functions use **different query targets** than the capstone — scheduled tasks, event logs, and script file deployment. The capstone requires you to **adapt these same patterns** to process queries, service queries, and Sysmon deployment.

Learning Objectives

- Build reusable functions that query remote systems and return structured results. **(K8, K9)**
- Use PowerShell remoting to perform remote deployment actions. **(K8)**
- Handle per-host failures without crashing the full workflow. **(K8)**
- Use a simple parallel orchestration pattern from Ops to the four remote targets. **(K8)**
- Recognize how existing function patterns can be repurposed for new tasks. **(K9)**

Chapter Map

Section	Focus
5.1	Remote query pattern
5.2	Build Function 1 — remote scheduled task query
5.3	Build Function 2 — remote event log query
5.4	Build Function 3 — deploy a script file remotely
5.5	Error handling and validation
5.6	Module assembly
5.7	Orchestrator pattern
5.8	Parallel execution from Ops
5.9	Troubleshooting and cleanup
5.10	Chapter review and capstone readiness

 Key principle

The functions you build in this chapter are **not** the capstone functions. They teach the same patterns using different targets. The capstone requires you to apply these patterns to process inventory, service inventory, and Sysmon deployment.

5.1 – Remote Query Pattern

Chapter 5 › 5.1 – Remote Query Pattern

5.1 – Remote Query Pattern

Every remote query function follows the same structure. Learn it once, and you can build any number of query functions by changing what runs inside the script block.

The pattern

1. Accept input by parameter
2. Use `Invoke-Command` with `-ErrorAction Stop`
3. On success: return `[pscustomobject]` with `Status = 'Success'`
4. On failure: return `[pscustomobject]` with `Status = 'Failed'`

```

function Get-RemoteExample {
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            # ← your query goes here
            hostname
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteExample'
            Status        = 'Success'
            Data          = $Data
            Error          = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteExample'
            Status        = 'Failed'
            Data          = $null
            Error          = $_.Exception.Message
        }
    }
}

```

★ CAPSTONE CONNECTION

The capstone rubric (R1) requires two Get-* functions that accept -ComputerName , return [pscustomobject] , and work against multiple remote hosts. This pattern satisfies all of those requirements.

5.2 – Build Function 1: Remote Scheduled Task Query

Chapter 5 › 5.2 – Build Function 1: Remote Scheduled Task Query

5.2 – Build Function 1: Remote Scheduled Task Query

Your first practice function queries **scheduled tasks** from a remote target. Same pattern from 5.1, different data source.

```
function Get-RemoteTaskInfo {
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            Get-ScheduledTask | Where-Object State -eq 'Ready' |
                Select-Object TaskName, State, TaskPath -First 10
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteTaskInfo'
            Status        = 'Success'
            Data          = $Data
            Error         = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteTaskInfo'
            Status        = 'Failed'
            Data          = $null
            Error         = $_.Exception.Message
        }
    }
}
```

★ CAPSTONE CONNECTION

For the capstone, you will build `Get-RemoteProcessInventory` using this same pattern. Instead

of `Get-ScheduledTask` , you will use `Get-Process` inside the script block.

5.3 – Build Function 2: Remote Event Log Query

Chapter 5 › 5.3 – Build Function 2: Remote Event Log Query

5.3 – Build Function 2: Remote Event Log Query

Your second practice function queries **recent event log entries**. Same pattern — different data source.

```
function Get-RemoteEventSummary {
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            Get-WinEvent -LogName System -MaxEvents 10 |
                Select-Object TimeCreated, Id, LevelDisplayName, Message
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteEventSummary'
            Status        = 'Success'
            Data          = $Data
            Error         = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteEventSummary'
            Status        = 'Failed'
            Data          = $null
            Error         = $_.Exception.Message
        }
    }
}
```

Two functions built. Both use the same skeleton. The only difference is the script block content.
The pattern is reusable.

 CAPSTONE CONNECTION

For the capstone, you will build `Get-RemoteServiceInventory` . Instead of `Get-WinEvent` , you will use `Get-Service` .

5.4 – Build Function 3: Deploy Remote Support Files

Chapter 5 › 5.4 – Build Function 3: Deploy Remote Support Files

This function stages the same two files used later in the capstone: the Sysmon binary and the Sysmon configuration file. The goal here is to practice the deployment pattern cleanly without adding unnecessary structure.

There are five steps in the pattern: validate the local source paths, create the remote session, copy the files, verify that the files arrived in the expected destination, and close the session. That is enough for this lesson. It keeps the function readable and keeps the focus on the workflow students will actually need.

Function pattern

```
function Deploy-RemoteSupportFiles {
    [CmdletBinding()]
    param(
        [string]$ComputerName,
        [pscredential]$Credential,
        [string]$BinaryPath,
        [string]$ConfigPath
    )

    $Session = $null

    try {
        if (-not (Test-Path -Path $BinaryPath)) {
            throw "BinaryPath not found: $BinaryPath"
        }

        if (-not (Test-Path -Path $ConfigPath)) {
            throw "ConfigPath not found: $ConfigPath"
        }

        $Session = New-PSSession -ComputerName $ComputerName -Credential $Credential -
ErrorAction Stop

        Copy-Item -Path $BinaryPath -Destination 'C:\Windows\Temp\Sysmon64.exe' -ToSession
$Session -Force -ErrorAction Stop
        Copy-Item -Path $ConfigPath -Destination 'C:\Windows\Temp\sysmonconfig-export.xml'
-ToSession $Session -Force -ErrorAction Stop

        $Data = Invoke-Command -Session $Session -ScriptBlock {
            Get-Item 'C:\Windows\Temp\Sysmon64.exe', 'C:\Windows\Temp\sysmonconfig-
export.xml' |
                Select-Object Name, Length, LastWriteTime
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Deploy-RemoteSupportFiles'
            Status        = 'Success'
            Data          = $Data
            Error         = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Deploy-RemoteSupportFiles'
            Status        = 'Failed'
        }
    }
}
```

```

        Data      = $null
        Error      = $_.Exception.Message
    }
}
finally {
    if ($Session) {
        Remove-PSSession -Session $Session
    }
}
}

```

Why this version is cleaner

This version uses one `try`, one `catch`, and one `finally`. That is easier to follow and still guarantees that the session is removed if it was created.

Test the function

The example below keeps the source paths aligned to the capstone so the student sees the same file locations throughout the course.

```

$Credential = Get-Credential

Deploy-RemoteSupportFiles `
    -ComputerName 'System-1' `
    -Credential $Credential `
    -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
    -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'

```

What success looks like

A successful run should return a custom object showing the target computer, a success status, and file details for both staged files. That gives you a structured result you can later collect from the orchestrator without changing the pattern.

Capstone bridge

The capstone adds the install step after the copy succeeds. The deployment logic does not need to become more complicated first. Students should be comfortable with validation, session creation, file transfer, verification, and cleanup before they add installer execution.

5.5 – Error Handling and Validation

Chapter 5 › 5.5 – Error Handling and Validation

5.5 – Error Handling and Validation

Validate required files and inputs **before** launching remote actions. A clean function should fail early when the required local path, remote target, or input value is missing.

```
if (-not (Test-Path $ScriptPath)) {  
    throw "Source script not found: $ScriptPath"  
}
```

Both success and failure paths should return the same object shape. This makes output easier to read, compare, export, and troubleshoot.

```
# Success output  
[pscustomobject]@{  
    ComputerName = $ComputerName  
    Status       = 'Success'  
    Error        = $null  
}  
  
# Failure output  
[pscustomobject]@{  
    ComputerName = $ComputerName  
    Status       = 'Failed'  
    Error        = $_.Exception.Message  
}
```

✓ Practical standard

If a reviewer cannot tell the difference between your success and failure output shapes by reading your code, your error handling needs work.

5.6 – Module Assembly

Chapter 5 › 5.6 – Module Assembly

Once the practice functions work individually, the next step is to package them so they can be imported and reused as a single unit. That is the purpose of the module. In this course, the module is the container for your reusable tools, while the orchestrator is the script that decides when and where those tools run.

That distinction matters. A function belongs in the module because it represents a reusable capability. The orchestrator belongs in a separate script because its purpose is coordination: define the targets, gather the credential, import the module, call the functions, and collect the results. Keeping those roles separate makes the code easier to test, easier to update, and much easier to reason about once parallel jobs are introduced.

What the module is doing

- It stores the reusable functions in one file.
- It exposes only the commands you intend to call from the outside.
- It gives the orchestrator one import point instead of requiring function definitions to be copied into every script.
- It gives each background job a clean way to load the same tools when parallel execution begins.

Working model

Think of the module as the toolbox and the orchestrator as the work plan. The toolbox holds the tools. The work plan decides which tool runs, against which host, and in what order.

Example module

The example below shows a complete practice module. Notice that the file does not execute anything on its own. It only defines functions and exports them. That is exactly what you want from a reusable module. The deployment function also stays aligned to the same Sysmon source paths used later in the capstone.

```

# RemoteOpsTools.psm1

function Get-RemoteTaskInfo {
    [CmdletBinding()]
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            Get-ScheduledTask |
                Where-Object State -eq 'Ready' |
                Select-Object -First 10 TaskName, State, TaskPath
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteTaskInfo'
            Status        = 'Success'
            Data          = $Data
            Error         = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteTaskInfo'
            Status        = 'Failed'
            Data          = $null
            Error         = $_.Exception.Message
        }
    }
}

function Get-RemoteEventSummary {
    [CmdletBinding()]
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            Get-WinEvent -LogName System -MaxEvents 25 |
                Where-Object LevelDisplayName -in 'Error', 'Warning' |
                Select-Object -First 10 TimeCreated, Id, ProviderName, LevelDisplayName,

```

Message

```

    } -ErrorAction Stop

    [pscustomobject]@{
        ComputerName = $ComputerName
        Function      = 'Get-RemoteEventSummary'
        Status        = 'Success'
        Data          = $Data
        Error          = $null
    }
}
catch {
    [pscustomobject]@{
        ComputerName = $ComputerName
        Function      = 'Get-RemoteEventSummary'
        Status        = 'Failed'
        Data          = $null
        Error          = $_.Exception.Message
    }
}

function Deploy-RemoteSupportFiles {
    [CmdletBinding()]
    param(
        [string]$ComputerName,
        [pscredential]$Credential,
        [string]$BinaryPath,
        [string]$ConfigPath
    )

    $Session = $null

    try {
        if (-not (Test-Path -Path $BinaryPath)) {
            throw "BinaryPath not found: $BinaryPath"
        }

        if (-not (Test-Path -Path $ConfigPath)) {
            throw "ConfigPath not found: $ConfigPath"
        }

        $Session = New-PSSession -ComputerName $ComputerName -Credential $Credential -
ErrorAction Stop

        Copy-Item -Path $BinaryPath -Destination 'C:\Windows\Temp\Sysmon64.exe' -ToSession
$Session -Force -ErrorAction Stop
        Copy-Item -Path $ConfigPath -Destination 'C:\Windows\Temp\sysmonconfig-export.xml'
-ToSession $Session -Force -ErrorAction Stop
    }
}

```

```

        $Data = Invoke-Command -Session $Session -ScriptBlock {
            Get-Item 'C:\Windows\Temp\Sysmon64.exe', 'C:\Windows\Temp\sysmonconfig-
export.xml' |
                Select-Object Name, Length, LastWriteTime
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Deploy-RemoteSupportFiles'
            Status        = 'Success'
            Data          = $Data
            Error         = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Deploy-RemoteSupportFiles'
            Status        = 'Failed'
            Data          = $null
            Error         = $_.Exception.Message
        }
    }
    finally {
        if ($Session) {
            Remove-PSSession -Session $Session
        }
    }
}

Export-ModuleMember -Function Get-RemoteTaskInfo, Get-RemoteEventSummary, Deploy-
RemoteSupportFiles

```

Why the export list matters

The export list defines the module interface. When the module is imported, those are the commands you expect to be available in the session. This keeps the public surface of the module clear and prevents helper code or experimental functions from being exposed accidentally.

In a small training module the export list may feel optional, but it becomes more important as the function count grows. A clear export list also helps you confirm quickly whether the right module version is loaded.

Import and verify

Before moving on to the orchestrator, import the module directly and confirm that the expected commands appear. Then call one function to prove that the module is not only loading but also working.

```
$ModulePath = (Resolve-Path .\RemoteOpsTools.psm1).Path

Import-Module $ModulePath -Force
Get-Command -Module RemoteOpsTools

$Credential = Get-Credential

Deploy-RemoteSupportFiles `
    -ComputerName 'System-1' `
    -Credential $Credential `
    -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
    -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

Verification checkpoint

If the import succeeds but the commands are missing, inspect the function names and the `Export-ModuleMember` line first. If the commands appear but fail when called, test the function itself before moving to the orchestrator.

How this connects to jobs

The module is also what makes the job pattern manageable. A background job does not automatically inherit the functions that are already loaded in the parent session. Each job starts in its own runspace, so the module has to be imported again inside the job script block.

```
Start-Job -ArgumentList $ModulePath, $ComputerName, $Credential, $BinaryPath, $ConfigPath
-ScriptBlock {
    param($ModulePath, $ComputerName, $Credential, $BinaryPath, $ConfigPath)

    Import-Module $ModulePath -Force

    Deploy-RemoteSupportFiles `
        -ComputerName $ComputerName `
        -Credential $Credential `
        -BinaryPath $BinaryPath `
        -ConfigPath $ConfigPath
}
```

That is the practical reason the module section cannot be treated as just file organization. Without the module, the parallel pattern becomes clumsy very quickly. You would either have to redefine the functions inside each job or accept brittle execution flow. With the module in place,

each job can load the same command set in a predictable way.

Module versus orchestrator

- **Module:** reusable functions only
- **Orchestrator:** target list, credential capture, import, execution order, and result collection
- **Job script block:** import the module again and run the exported functions for its assigned target

Capstone bridge

In the capstone, the exact function bodies will change, but the relationship does not. The module remains the reusable command set. The orchestrator remains the script that coordinates how those commands are applied across multiple hosts.

5.7 – Orchestrator Pattern

Chapter 5 › 5.7 – Orchestrator Pattern

5.7 – Orchestrator Pattern

The orchestrator is a coordinator. It imports the module, builds the target list, prompts for credentials, launches work, and collects results. It should be **readable and short**.

```
$Targets = 'System-1','System-2','System-3','Depot'
$Credential = Get-Credential

Import-Module .\RemoteOpsTools.psm1 -Force

# Run against one target first to test
Get-RemoteTaskInfo -ComputerName 'System-1' -Credential $Credential
```

✓ Practical standard

Keep the orchestrator readable. If it is longer than 30–40 lines, you have probably put too much logic in it.

5.8 – Parallel Execution from Ops

Chapter 5 › 5.8 – Parallel Execution from Ops

5.8 – Parallel Execution from Ops

Once functions work one target at a time, the orchestrator can fan out work in parallel.

```
$Targets = 'System-1','System-2','System-3','Depot'
$Credential = Get-Credential
$ModulePath = (Resolve-Path .\RemoteOpsTools.psm1).Path

Import-Module $ModulePath -Force

$Jobs = foreach ($Target in $Targets) {
    Start-Job -ArgumentList $Target, $Credential, $ModulePath -ScriptBlock {
        param($ComputerName, $Credential, $ModulePath)
        Import-Module $ModulePath -Force
        Get-RemoteTaskInfo -ComputerName $ComputerName -Credential $Credential
    }
}

$Results = $Jobs | Receive-Job -Wait -AutoRemoveJob
$Results | Format-Table -AutoSize
```

Counting data returned by wrapper objects

If a function returns a wrapper object with a `Data` property, count the items inside `Data`. Do not count the wrapper object itself.

```
# Correct: count the inventory data inside the wrapper object
ProcessCount = $ProcessResults.Data.Count
ServiceCount = $ServiceResults.Data.Count
```

Watch the object shape

`$ProcessResults.Data.Count` counts the wrapper object. In PowerShell 7, a single scalar object reports a count of `1`, even when the real inventory list contains many items.

5.9 – Troubleshooting and Cleanup

Chapter 5 › 5.9 – Troubleshooting and Cleanup

5.9 – Troubleshooting and Cleanup

When testing repeatedly, sessions and jobs accumulate. Make cleanup a habit.

```
Get-PSSession | Remove-PSSession
Get-Job | Remove-Job -Force
```

Check	Command
Test credentials separately	<code>Invoke-Command -ComputerName System-1 -Credential \$Cred -ScriptBlock { whoami }</code>
Confirm files exist on Ops	<code>Test-Path \$BinaryPath</code>
Confirm remote host is reachable	<code>Test-WSMan System-1</code>
Read failure text from object	<code>\$Results Where-Object Status -eq 'Failed'</code>
Check orphaned sessions	<code>Get-PSSession</code>
Check orphaned jobs	<code>Get-Job</code>

 **Common mistake – accumulating stale sessions**
If you test 10 times without cleanup, you may have 10 open sessions to each target. Run `Get-PSSession | Remove-PSSession` between test runs.

5.10 – Chapter Review and Capstone Readiness

You are ready for the capstone when you can do the following with confidence:

- Build a remote query function using the pattern from 5.1
- Build a remote deployment function using the pattern from 5.4
- Return structured output from every function
- Handle per-host errors without crashing the workflow
- Assemble functions into a module and verify the import
- Write an orchestrator that coordinates work across all targets
- Execute work in parallel using Start-Job

The capstone requires you to adapt, not copy

Capstone function	Chapter 5 equivalent	What changes
Get-RemoteProcessInventory	Get-RemoteTaskInfo	Script block: Get-Process instead of Get-ScheduledTask
Get-RemoteServiceInventory	Get-RemoteEventSummary	Script block: Get-Service instead of Get-WinEvent
Install-SysmonRemote	Deploy-RemoteSupportFiles	Two copies (binary+config), file verification, clean session cleanup

Chapter 5 Exercise — End-to-End Dry Run

Use your Chapter 5 functions to validate the full workflow.

1. Import your RemoteOpsTools.psm1 module. Verify all three functions loaded.
2. Run each function against one target. Confirm structured output.
3. Run all three in parallel against all four targets.
4. Clean up: Get-PSSession | Remove-PSSession and Get-Job | Remove-Job -Force .

Success criteria: All functions return structured output. Parallel execution works. Per-host failures are captured.

Chapter 6: Capstone

Chapter Purpose

This chapter brings the course together into one final build: a reusable tools module and an orchestrator script that run from Ops against the remote targets in the domain range. The goal is not to introduce a new pattern. The goal is to apply the patterns from the earlier chapters cleanly, consistently, and with enough discipline that the final result can be tested, explained, and maintained.

By the time you reach the capstone, you have already worked through the pieces that matter most: understanding returned objects, shaping output, writing functions, handling remote execution with PowerShell remoting, packaging functions in a module, and using jobs when work needs to scale across multiple systems. Chapter 6 shifts the focus from *learning individual techniques* to *assembling them into one coherent solution*.

In this course, the student workstation is **Ops**. The remote targets are **System-1**, **System-2**, **System-3**, and **Depot**. The range is domain-based, so the workflow should assume normal domain credential use rather than ad hoc local-account workarounds. That changes the environment, but it does not change the standard: the script still needs to be readable, dependable, and grounded in the PowerShell patterns practiced earlier.

What the capstone should demonstrate

A successful capstone shows more than simple command execution. It shows that the script author understands how to break a problem into functions, how to pass parameters instead of hard-coding choices, how to return structured output, how to handle remote work without turning the script into a tangle of one-off commands, and how to verify the result before declaring success.

Deliverable	What it should show
.psm1 tools module	Reusable functions with consistent parameters, structured output, and clean export behavior
.ps1 orchestrator	A readable control script that runs from Ops, prompts once for credentials, and coordinates the work
Two Get-* functions	Remote queries that return useful inventory data from the target systems
One deployment function	A controlled remote install of Sysmon using the supplied binary and configuration file

Chapter map

Section	Focus
6.1	Capstone requirements
6.2	Module standard
6.3	Required functions
6.4	Orchestrator standard
6.5	Parallel execution standard
6.6	Error handling standard
6.7	Validation and testing guide
6.8	Submission checklist

✓ Reading approach

Treat this chapter as the build standard for the final exercise. Read it once for the big picture, then use each section as a reference while you implement and test your own module and orchestrator.

6.1 – Capstone Requirements

The capstone is intentionally narrow. That is a strength, not a limitation. The final build is supposed to prove that you can apply the course patterns to a realistic administrative task set without burying the solution under unnecessary complexity.

The solution runs from **Ops** and targets **System-1, System-2, System-3**, and **Depot**. Your deliverables are a PowerShell module named consistently for the capstone and an orchestrator script that imports that module and uses its exported functions. The remote work should use PowerShell remoting. In this course, that means building around `New-PSSession`, `Invoke-Command`, and the patterns you practiced earlier.

Minimum required behavior

- Create a module that contains exactly the capstone functions you intend to use.
- Include two remote inventory functions: `Get-RemoteProcessInventory` and `Get-RemoteServiceInventory`.
- Include one remote deployment function: `Install-SysmonRemote`.
- Use the supplied Sysmon files at `C:\Tools\Sysmon\Sysmon64.exe` and `C:\Tools\Sysmon\sysmonconfig-export.xml` in your examples and testing workflow.
- Return structured output rather than writing status text with `Write-Host`.
- Use one credential prompt at the orchestrator level and pass that credential into the work that follows.
- Support one target during early testing and all four targets during final execution.

Practical standard

The capstone does not need a self-healing framework, persistent credential vaulting, TrustedHosts reconfiguration logic, or multiple fallback remoting methods. A clean solution that reliably uses the taught pattern is stronger than a sprawling solution that tries to solve every possible environment problem.

Expected execution model

The orchestration flow should be easy to explain in plain language: prompt for credentials once, import the module, run the required functions against the target systems, collect the results, and present those results in a way that makes success or failure obvious. If another student or instructor cannot follow the flow quickly by reading the script top to bottom, the design is probably carrying too much overhead.

```

$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$ModulePath = (Resolve-Path .\CapstoneTools.psm1).Path
$Credential = Get-Credential
Import-Module $ModulePath -Force
Get-RemoteProcessInventory -ComputerName 'System-1' -Credential $Credential
Get-RemoteServiceInventory -ComputerName 'System-1' -Credential $Credential
Install-SysmonRemote -ComputerName 'System-1' -Credential $Credential `
    -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
    -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'

```

That example is not the finished capstone. It shows the operating model. Before parallel execution or full orchestration enters the picture, each individual function should work cleanly against one target. That staging matters because it keeps troubleshooting focused. When something fails, you want to know whether the problem is in the function itself or in the orchestration wrapped around it.

What strong submissions usually have in common

Characteristic	What it looks like in practice
Consistent parameters	Each function accepts at least <code>-ComputerName</code> and <code>-Credential</code> in a predictable way
Structured return values	Each function returns objects that identify the target and the result clearly
Readable control flow	The orchestrator is short enough to understand without hunting through nested logic
Tidy cleanup	Sessions, jobs, and temporary artifacts are cleaned up deliberately
Incremental testing	The student proves one target first, then expands to all required targets

What to avoid

Do not treat the capstone as a contest to write the most complicated script. Nested remoting patterns, mixed remoting technologies in the same function, or deep layers of recovery logic usually make the solution harder to validate and harder to explain.

6.2 – Module Standard

The module is the reusable part of the capstone. It should contain the functions that do the real work while the orchestrator stays focused on coordination. That separation matters because it makes testing easier. You can import the module and test one function at a time before you ever run the full `.ps1` file.

A good module reads like a small toolset. The exported functions are clear, the parameter sets are predictable, and nothing important happens merely because the module was imported. Importing a module should load capability. It should not silently launch work.

What belongs in the module

- Function definitions
- Parameter validation that supports the function contract
- Structured return objects
- Session creation and cleanup inside the functions that need it
- An explicit export statement for the functions the orchestrator should call

What should stay out of the module

- Interactive prompts that fire on import
- Long blocks of one-time testing code
- Hard-coded credentials
- Ad hoc transcript text written with Write-Host as the primary result
- Top-level commands that execute automatically when the module is imported

```
function Get-RemoteProcessInventory {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$ComputerName,
        [Parameter(Mandatory)]
        [pscredential]$Credential
    )
    # Function body
}
function Get-RemoteServiceInventory {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$ComputerName,
        [Parameter(Mandatory)]
        [pscredential]$Credential
    )
    # Function body
}
function Install-SysmonRemote {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$ComputerName,
        [Parameter(Mandatory)]
        [pscredential]$Credential,
        [Parameter(Mandatory)]
        [string]$BinaryPath,
        [Parameter(Mandatory)]
        [string]$ConfigPath
    )
    # Function body
}
Export-ModuleMember -Function Get-RemoteProcessInventory, Get-RemoteServiceInventory,
Install-SysmonRemote
```

Why the parameter pattern matters

Keeping the parameter pattern consistent reduces friction everywhere else in the build. When each function accepts `-ComputerName` and `-Credential` in the same way, the orchestrator can call them predictably and the student can troubleshoot them predictably. In practice, consistency is one of the fastest ways to keep a capstone from becoming messy.

Design choice	Recommended standard	Reason
Function names	Use verb-noun names such as <code>Get-RemoteProcessInventory</code>	The purpose is obvious before the function is opened
Parameter names	Keep shared parameters identical across functions	The orchestrator can call them without special-case logic
Return type	Return PowerShell objects	Objects can be filtered, counted, exported, and tested
Export behavior	Use <code>Export-ModuleMember</code>	Only the intended public functions are exposed

Module mindset

Think of the module as the part you would want to keep after the course ends. If the orchestrator were deleted, the module should still contain functions that make sense and can be tested on their own.

Import and verify before building the orchestrator

```
Import-Module .\CapstoneTools.psm1 -Force
Get-Command -Module CapstoneTools
```

If the import does not expose the functions you expect, stop there and fix the module first. Troubleshooting a failed orchestrator when the module import is already wrong wastes time and hides the real issue.

Common source of confusion

A function existing in the file is not the same thing as the function being available after import. Missing export statements, syntax errors, or accidental top-level code can make a module behave very differently from what the file appears to contain.

6.3 – Required Functions

The capstone requires three functions, but the deeper requirement is that each one should have a clear job and a clear output contract. A reader should be able to identify what a function is supposed to do by its name, its parameter block, and the object shape it returns.

1) Get-RemoteProcessInventory

This function should connect to the remote system, retrieve process information, and return inventory data as objects. The point is not to dump every property available from `Get-Process`. The point is to choose a useful set of properties and return them consistently.

```
Get-RemoteProcessInventory -ComputerName 'System-1' -Credential $Credential |  
Select-Object ComputerName, Name, Id, CPU
```

Recommended mindset

Choose properties that help you recognize what is running and where the record came from. Including the source system in the returned object is especially important once you move to multiple targets.

2) Get-RemoteServiceInventory

This function should connect to the remote system, retrieve service information, and return objects that make service state easy to review. Again, the goal is not maximum property volume. The goal is useful, readable inventory data.

```
Get-RemoteServiceInventory -ComputerName 'System-1' -Credential $Credential |  
Select-Object ComputerName, Name, DisplayName, Status, StartType
```

3) Install-SysmonRemote

This function is the deployment function. It should validate the provided file paths, create a remote session, copy the Sysmon binary and configuration file to the target, execute the installer, verify the install, and return a summary object. The capstone path examples should remain aligned to the course standard: `C:\Tools\Sysmon\Sysmon64.exe` and `C:\Tools\Sysmon\sysmonconfig-export.xml`.

```
Install-SysmonRemote -ComputerName 'System-1' -Credential $Credential `
  -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
  -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

Think in function contracts

One of the easiest ways to strengthen a capstone solution is to define what success and failure should look like before writing the full function body. That keeps the function from drifting into vague behavior.

Function	Should accept	Should return
Get-RemoteProcessInventory	-ComputerName , -Credential	One or more objects describing process inventory from the target system
Get-RemoteServiceInventory	-ComputerName , -Credential	One or more objects describing service inventory from the target system
Install-SysmonRemote	-ComputerName , -Credential , -BinaryPath , -ConfigPath	A summary object that clearly indicates success or failure for the installation attempt

A clean summary object for the deployment function

```
[pscustomobject]@{
  ComputerName = $ComputerName
  Function = 'Install-SysmonRemote'
  Status = 'Success'
  Details = 'Sysmon installed and verified'
}
```

That return pattern is simple on purpose. It is easy to read in the console, easy to export later if needed, and easy to compare across multiple systems. When something fails, the same shape can be returned with a different status and a meaningful error message.

✅ Strong capstone habit

Keep each function responsible for one category of work. Inventory functions should gather and return data. The deployment function should perform the install workflow. Once functions begin mixing unrelated tasks, they become harder to test and harder to trust.

⚠️ Do not overbuild the deployment function

The deployment function does not need to manage every possible environmental exception. It should handle the taught workflow cleanly, return a usable result, and clean up after itself. That is the standard to aim for.

6.4 – Orchestrator Standard

The orchestrator is the control script. It should not contain the deepest technical work. Its job is to set the run conditions, prompt for credentials, import the module, call the exported functions in a sensible sequence, and present the results.

A good orchestrator reads from top to bottom without surprises. You should be able to identify the input parameters, the module path, the target systems, the credential collection point, and the work sequence without scrolling through dense nested logic.

Recommended responsibilities

- Accept the target systems and module path as parameters
- Prompt once for credentials with Get-Credential
- Resolve and import the module
- Run the required module functions
- Collect and display results
- Clean up background jobs if the script uses parallel execution

```
[CmdletBinding()]  
param(  
    [string[]]$ComputerName = @('System-1', 'System-2', 'System-3', 'Depot'),  
    [string]$ModulePath = (Join-Path $PSScriptRoot 'CapstoneTools.psm1')  
)  
$Credential = Get-Credential  
$ResolvedModulePath = (Resolve-Path $ModulePath).Path  
Import-Module $ResolvedModulePath -Force  
# Orchestration logic follows
```

Why the credential prompt belongs here

The orchestrator is the right place to collect credentials because it sits at the start of the run and can pass that credential object into the rest of the work. That keeps the module reusable and avoids repeated prompts. It also avoids a common failure pattern: trying to prompt for credentials from inside background jobs or nested remote commands.

A readable sequence

```
$Targets = $ComputerName
foreach ($Target in $Targets) {
    Get-RemoteProcessInventory -ComputerName $Target -Credential $Credential
    Get-RemoteServiceInventory -ComputerName $Target -Credential $Credential
    Install-SysmonRemote -ComputerName $Target -Credential $Credential `
        -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
        -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
}
```

That single-threaded version is worth building first even if your final standard uses jobs. It is easier to read, easier to debug, and easier to validate. Once the sequential version works, you can change the control structure to parallel execution without changing the function contracts.

Part of script	Keep it focused on
Parameter block	Inputs to the run
Credential collection	One prompt at the start
Module import	Loading the reusable tools
Execution block	Calling the required functions in a clear order
Result handling	Presenting what happened without hiding errors

 A useful design test

If you can explain the orchestrator in five or six short sentences, the structure is probably in good shape. If the explanation turns into a tour of exceptions, special cases, and workarounds, the script likely needs to be simplified.

 Keep one-time commands out of the final script

Commands used only for ad hoc troubleshooting, such as exploratory remoting tests, should be used during validation but should not stay embedded in the final orchestrator unless they are part of the intended execution flow.

6.5 – Parallel Execution Standard

Parallel execution is useful here because the capstone targets multiple remote systems. The gain is not just speed. Parallel execution also gives you a cleaner model for "do the same work on each system" when the functions are already built and tested.

In this course, the practical way to introduce parallel work is with background jobs. Each job receives the module path, the target system name, and the credential object. Inside the job, the module is imported again because each background job starts in a separate session state.

Important detail

If the orchestrator uses jobs, do not call `Get-Credential` inside the job. Prompt once before the jobs are created, then pass the credential object into each job with `-ArgumentList`.

Parallel pattern

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$Credential = Get-Credential
$ModulePath = (Resolve-Path .\CapstoneTools.psm1).Path
$Jobs = foreach ($ComputerName in $Targets) {
    Start-Job -Name $ComputerName -ArgumentList $ModulePath, $ComputerName, $Credential -
ScriptBlock {
    param($ModulePath, $ComputerName, $Credential)
    Import-Module $ModulePath -Force
    $ProcessResults = Get-RemoteProcessInventory -ComputerName $ComputerName -
Credential $Credential
    $ServiceResults = Get-RemoteServiceInventory -ComputerName $ComputerName -
Credential $Credential
    $DeployResult = Install-SysmonRemote -ComputerName $ComputerName -Credential
$Credential `
    -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
    -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
    [pscustomobject]@{
        ComputerName = $ComputerName
        ProcessCount = $ProcessResults.Data.Count
        ServiceCount = $ServiceResults.Data.Count
        DeployStatus = $DeployResult.Status
    }
}
}
}
>null = $Jobs | Wait-Job
$Results = $Jobs | Receive-Job
$Jobs | Remove-Job
$Results
```

Why this pattern works

Element	Reason it matters
Start-Job	Creates one unit of work per target system
-ArgumentList	Passes the values the job needs without relying on outer scope
Import-Module inside the job	Each job has its own runspace and must load the module for itself
Wait-Job and Receive-Job	Keeps collection of results orderly
Remove-Job	Cleans up after the run

Notice that the job does not attempt to become an orchestrator inside the orchestrator. It imports the module, calls the required functions, and returns a small summary object. That keeps the result compact and keeps the parallel block readable.

Build in this order

1. Make each function work against one target.
2. Make the orchestrator work sequentially against one target.
3. Expand the sequential run to all required targets.
4. Introduce background jobs only after the previous steps are clean.
5. Verify that the job results still return the information you expect.

Where students usually lose time

Most job-related problems are not job problems. They are missing module imports inside the job, missing arguments in `-ArgumentList`, or attempts to use interactive prompts from inside the job.

Reasonable standard

A compact job wrapper around already-tested functions is enough. The capstone does not need nested jobs, threaded runspace frameworks, or a custom job-monitoring system.

6.6 – Error Handling Standard

Error handling should make the capstone easier to trust, not harder to read. The clean standard is to use one clear try/catch/finally pattern around the work that can fail, convert non-terminating problems into terminating ones where needed, and return a result that explains what happened.

This is one of the places where restraint matters. Deep nesting is rarely a sign of strength. In most capstone functions, a single outer try block is enough when the steps inside it use explicit error behavior and the cleanup is handled in finally.

```

function Install-SysmonRemote {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$ComputerName,
        [Parameter(Mandatory)]
        [pscredential]$Credential,
        [Parameter(Mandatory)]
        [string]$BinaryPath,
        [Parameter(Mandatory)]
        [string]$ConfigPath
    )
    $Session = $null
    try {
        if (-not (Test-Path $BinaryPath)) {
            throw "Binary path not found: $BinaryPath"
        }
        if (-not (Test-Path $ConfigPath)) {
            throw "Config path not found: $ConfigPath"
        }
        $Session = New-PSSession -ComputerName $ComputerName -Credential $Credential -
ErrorAction Stop
        Copy-Item $BinaryPath -Destination 'C:\Windows\Temp\Sysmon64.exe' -ToSession
$Session -Force -ErrorAction Stop
        Copy-Item $ConfigPath -Destination 'C:\Windows\Temp\sysmonconfig-export.xml' -
ToSession $Session -Force -ErrorAction Stop
        Invoke-Command -Session $Session -ScriptBlock {
            & 'C:\Windows\Temp\Sysmon64.exe' -accepteula -i 'C:\Windows\Temp\sysmonconfig-
export.xml'
        } -ErrorAction Stop
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function = 'Install-SysmonRemote'
            Status = 'Success'
            Details = 'Sysmon install completed'
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function = 'Install-SysmonRemote'
            Status = 'Failed'
            Details = $_.Exception.Message
        }
    }
    finally {
        if ($Session) {
            Remove-PSSession $Session
        }
    }
}

```

```
}
}
```

What this pattern gives you

Feature	Benefit
Path checks before remoting	Simple local problems are caught before a session is opened
-ErrorAction Stop	Cmdlets that would otherwise continue can be handled in the catch block
Single catch block	Failure handling stays readable and consistent
Finally cleanup	The session is removed whether the function succeeds or fails
Structured failure object	The caller can still work with the result programmatically

When nested try blocks are usually unnecessary

Sometimes students start nesting try blocks around every individual step. That usually makes the function harder to read without improving the result. In a capstone of this size, nested try blocks are only justified when different failures truly need different recovery actions. If the handling is the same—return a failure object and clean up—one outer structure is usually the better design.

Helpful testing habit

During validation, force a known failure once. Use a bad file path or an unavailable target and confirm that the function returns a clean failure object instead of breaking the entire script flow in a confusing way.

Avoid these patterns

- Using `Write-Host` as the only error output
- Returning plain strings for failure when success returns objects
- Leaving sessions open when an exception occurs
- Catching an error and then hiding the useful message

6.7 – Validation and Testing Guide

Validation should move from simple to complex. The fastest way to burn time in the capstone is to test everything at once. A better sequence is to verify the environment, test the module import, test remoting, test each function against one target, and only then run the orchestrator across multiple targets.

Phase 1: Verify the local prerequisites on Ops

```
Test-Path .\CapstoneTools.psm1
Test-Path .\Invoke-Capstone.ps1
Test-Path 'C:\Tools\Sysmon\Sysmon64.exe'
Test-Path 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

At this phase, you are only proving that the expected files are present where the script expects them to be. If the file paths are wrong, fix that first.

Phase 2: Verify the module import

```
Import-Module .\CapstoneTools.psm1 -Force
Get-Command -Module CapstoneTools
```

The command list should show the three capstone functions. If it does not, stop and correct the module before moving on.

Phase 3: Verify remoting against one target

```
$Credential = Get-Credential
Invoke-Command -ComputerName 'System-1' -Credential $Credential -ScriptBlock {
    hostname
}
```

Reason for this step

This confirms that the environment can support the function calls that depend on remoting. It also separates basic remoting problems from function logic problems.

Phase 4: Unit-test each function

```
Get-RemoteProcessInventory -ComputerName 'System-1' -Credential $Credential
Get-RemoteServiceInventory -ComputerName 'System-1' -Credential $Credential
Install-SysmonRemote -ComputerName 'System-1' -Credential $Credential `
  -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
  -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

Each function should be proven individually. If one of them fails, fix that function first rather than masking the failure by moving into the orchestrator.

Phase 5: Run the orchestrator against one target

```
.\Invoke-Capstone.ps1 -ComputerName 'System-1' `
  -ModulePath (Resolve-Path .\CapstoneTools.psm1).Path
```

Phase 6: Expand to all required targets

```
.\Invoke-Capstone.ps1 -ComputerName 'System-1', 'System-2', 'System-3', 'Depot' `
  -ModulePath (Resolve-Path .\CapstoneTools.psm1).Path
```

Phase 7: Perform one deliberate failure test

```
Install-SysmonRemote -ComputerName 'System-1' -Credential $Credential `
  -BinaryPath 'C:\Tools\Sysmon\Missing.exe' `
  -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

That last test is valuable because it proves the function's failure handling. A clean failure result is part of a good solution.

Test level	Question being answered
File path checks	Are the required inputs present?
Module import	Are the exported functions available?
Basic remoting	Can Ops reach the target with the provided credential?
Function tests	Does each tool work on its own?
One-target orchestration	Does the control script call the tools correctly?
Full orchestration	Does the final workflow scale to all required systems?

 Efficient troubleshooting rule

When a higher-level test fails, move back down one layer. Do not keep testing the orchestrator when the individual function has already shown you where the issue lives.

6.8 – Submission Checklist

The last step before submission is not writing more code. It is confirming that the code you already wrote meets the standard you intended to meet. A short checklist is useful here because it prevents last-minute drift.

Final review checklist

- The module imports cleanly with `Import-Module .\CapstoneTools.psm1 -Force .`
- Only the intended capstone functions are exported.
- The required functions are present: `Get-RemoteProcessInventory`, `Get-RemoteServiceInventory`, and `Install-SysmonRemote`.
- The orchestrator prompts once for credentials with `Get-Credential`.
- The orchestrator accepts the remote targets and module path as parameters.
- The individual functions work against one target before the full run is attempted.
- The final run completes against System-1, System-2, System-3, and Depot.
- Returned results are structured and readable.
- Sessions and jobs are cleaned up.
- The script paths for the Sysmon binary and configuration file match the course standard.

A final command sequence

```
Import-Module .\CapstoneTools.psm1 -Force
Get-Command -Module CapstoneTools
.\Invoke-Capstone.ps1 -ComputerName 'System-1', 'System-2', 'System-3', 'Depot' `
    -ModulePath (Resolve-Path .\CapstoneTools.psm1).Path
```

What to look for in the final output

Area	What you want to see
Inventory results	Objects that clearly identify the source system and the returned data
Deployment results	A success or failure status with a useful details field
Overall readability	Output that can be explained quickly without digging through raw console noise
Script behavior	A predictable run with no hidden prompts and no leftover jobs or sessions

 Professional habit

A good submission is one you can defend line by line. If an instructor points at a function, a parameter, or a block in the orchestrator, you should be able to explain why it exists and what it returns.

That is the real finish line for the capstone. The code should run, but it should also be understandable, testable, and built from patterns you can reuse in later scripting work.